

光聖 (6442) AI 應用開發工程師

面試模擬題庫 —— 30 題 (含解答)

使用方式：解答以「深入淺出 + 比喻」呈現，但保留核心術語與觀念。建議先看題目自己組織答案，再對照參考解答；行為面題目 (Q24-Q30) 請填入自己的真實案例與數字。

A Agent 架構與核心觀念

Q1 什麼是 AI Agent ? 跟單純呼叫 LLM API 有什麼不同 ?

單純呼叫 LLM 像是「問一個很博學的朋友一個問題，他給你一個答案就結束」。Agent 則像派一個特助去辦事：他會自己拆解任務 (Planning)、判斷下一步該做什麼 (Reasoning)、需要時去查資料或操作工具 (Tool Use)、記得前面發生過什麼 (Memory)，並反覆循環直到把事辦完。核心差異在於 Agent 有「感知—決策—行動」的迴圈與自主性，而不是一次性的問答。

Q2 Reasoning、Planning、Memory 三者 in Agent 裡各扮演什麼角色 ?

用「請人去買菜煮一桌飯」比喻：Planning 是「先列出要做幾道菜、所以要買哪些料、先後順序」；Reasoning 是「到了現場發現某個食材沒貨，當下判斷用什麼替代」；Memory 是「記得這家人上次說不吃辣、冰箱裡還有半顆洋蔥」。Planning 管長期路徑、Reasoning 管當下決策、Memory 管前後文與經驗的延續——三者缺一，Agent 就會變成健忘又不會應變的工具。

Q3 ReAct 是什麼 ? 為什麼重要 ?

ReAct 是 Reasoning + Acting 交錯進行的框架。就像偵探辦案：不是悶頭想完才行動，而是「想一下 (這條線索可疑) → 行動 (去查監視器) → 看到結果再想 (果然有人) → 再行動」。它讓 Agent 把「思考」跟「用工具拿到的真實資訊」交替進行，大幅減少憑空亂編 (hallucination)，因為每一步推理都被真實的工具回傳結果校正。

Q4 Agent 容易陷入無限迴圈或卡住，你會怎麼處理 ?

就像派出去的特助鬼打牆、一直重複同一步。實務上要加「護欄」：設定最大步數上限、偵測重複動作就強制中斷、設計反思（reflection）機制讓它檢查「我是不是在原地打轉」、以及最後的 fallback（真的解不了就回報人類接手）。重點是不能讓自主性無限放大成失控。

B 記憶系統（公司明確點名 Neo4j / pgvector）

Q5 為什麼 Agent 需要記憶系統？短期記憶和長期記憶差在哪？

短期記憶就像「對話當下你腦中暫存的內容」（context window），對話一結束或太長就被擠掉。長期記憶像「筆記本」，把重要的事寫進外部資料庫，下次再撈出來。沒有長期記憶的 Agent，就像每天上班都失憶的員工——你得每次從頭教他公司怎麼運作。

Q6 向量資料庫（pgvector）跟知識圖譜（Neo4j）在記憶系統裡分別解決什麼問題？

向量庫擅長「語意相似」的模糊搜尋——像你只記得「那本講習慣養成的書」，它能幫你找到《原子習慣》。知識圖譜擅長「明確關係」的精準推理——像「跟客戶 A 有合作的供應商、且該供應商也供貨給競爭對手」這種多跳關聯查詢。製造業很多知識是「誰供誰、哪個製程影響哪個良率」這種關係網，所以圖譜特別有價值。實務上兩者常結合成 GraphRAG。

Q7 如果讓你設計這家公司的 Agent 記憶系統，你會怎麼分層？

我會分三層：對話層（短期 context，當前任務）、語意層（pgvector 存文件、規格、歷史案例，做模糊檢索）、關係層（Neo4j 存實體與關係，例如客戶—訂單—料件—製程的關聯）。查詢時先用圖譜鎖定相關實體，再用向量補語意細節——就像「先用地圖定位區域，再用放大鏡看細節」。

Q8 PostgreSQL + pgvector 相較專用向量庫（如 Milvus）的取舍？

pgvector 的好處是「一個資料庫搞定結構化資料 + 向量」，少維護一套系統，對剛起步、資料量不大的內部專案很劃算，像「一台多功能事務機 vs 專業印表機」。但資料量上千萬筆、要極致檢索效能時，專用向量庫（Milvus）的索引與分散式能力會更好。新導入階段我會傾向先用 pgvector 降低複雜度，預留之後遷移的彈性。

C 框架與工具

Q9 Dify、n8n、LangGraph 各適合什麼場景？

用「蓋房子的工具」比喻：Dify 像樣品屋——介面化、快速搭出聊天機器人或簡單 RAG，適合讓非工程師也能參與、快速 PoC。n8n 像樂高水管——把各種系統（郵件、資料庫、API）用拖拉方式串起自動化流程，整合既有系統很強。LangGraph 像鋼筋水泥的自建工程——用程式碼精準控制 Agent 的狀態流轉、迴圈、分支，適合複雜、需要客製化控制的正式系統。新導入時我會先用 Dify / n8n 快速驗證價值，核心複雜流程再用 LangGraph 落地。

Q10 為什麼 LangGraph 用「圖 (graph)」來描述 Agent，而不是傳統的鏈 (chain)？

Chain 像一條單行道， $A \rightarrow B \rightarrow C$ 走完就結束。但真實任務常需要「條件分支、回頭重做、迴圈」——像導航會根據路況重新規劃。Graph 用「節點 + 邊」描述狀態流轉，能自然表達「這步失敗就退回上一步」「滿足條件才往下」，更貼近 Agent 需要的自主決策與迴圈。

Q11 你用過 DSPy，它跟一般寫 prompt 有什麼不同？

傳統寫 prompt 像「手工反覆調整咒語」，憑經驗試。DSPy 把它變成「可優化的程式」——你定義輸入輸出的規格與評估指標，讓它自動幫你找出好的 prompt / few-shot 範例。比喻就是「從手調收音機頻率，變成自動掃台找最清晰的訊號」。對長期維護的系統，這種可優化、可量測的方式比手調 prompt 穩定得多。

D LLM 原理

Q12 簡單說明 LLM 為什麼會「幻覺」(hallucination)？

LLM 本質是「猜下一個字最可能是什麼」的機率機器，它追求「聽起來合理」而不是「事實正確」。就像一個很會講話、但不確定時也不肯說「我不知道」的人，會自信地編一個聽起來很順的答案。所以它不是在說謊，是「不知道自己不知道」。

Q13 RAG 怎麼緩解幻覺？它的限制是什麼？

RAG 就像考試從「閉卷」改成「開卷」——先去資料庫撈相關文件，再讓模型「根據這些資料回答」，答案就有了依據。但限制在於：如果撈錯資料（檢索品質差），開卷也會抄錯答案；資料庫沒有的東西它還是會掰。所以 RAG 的成敗，一半在「檢索」而不只在「生成」。

Q14 Temperature 參數調高調低分別代表什麼？什麼時候該用哪個？

Temperature 像「想像力旋鈕」。調低（接近 0）→ 回答保守、可重現，適合要精準的場景（查規格、做分類）。調高 → 回答有創意、多變，適合腦力激盪。在企業內部 Agent，多數要求穩定可靠，所以通常偏低。

Q15 Context window 是什麼？為什麼不能無限把資料塞進去？

Context window 是模型「一次能看進去的内容上限」，像人的短期記憶或桌面大小。塞太多會有三個問題：成本暴增（按 token 收費）、延遲變慢、以及「迷失在中間」（lost in the middle，太長時模型會忽略中段內容）。所以好的設計是「只把最相關的東西放桌上」，這正是 RAG 與記憶系統的价值。

E 評估、優化、穩定性（企業最在乎）

Q16 怎麼評估一個 Agent 做得好不好？

不能只憑「感覺它回答得不錯」。我會分層建指標：任務成功率（有沒有把事辦成）、正確性 / 忠實度（答案有沒有依據，可用 LLM-as-judge 或人工標註）、效率（步數、延遲、token 成本）、穩定性（同樣輸入會不會時好時壞）。就像評廚師不能只嚐一口，要看「出菜成功率、味道穩定度、用料成本」。建立可量化的評估基準（eval set），是長專案不退化的安全網。

Q17 LLM 輸出不穩定、同樣問題答案不一樣，企業要上線怎麼辦？

這是企業落地最現實的痛。對策：對需要結構化的輸出用 structured output / function calling 強制格式；用較低 temperature；對關鍵流程加驗證層（程式檢查輸出合不合規，不合就重試）；建立 eval + 回歸測試，每次改 prompt 或換模型都跑一遍，確保沒有退化。本質是「用工程手段把不確定性關進可控的籠子」。

Q18 如何優化 Agent 的成本與延遲？

幾個槓桿：模型分級（簡單任務用小模型、難的才呼叫大模型，像不需要每件事都找專家）、快取重複的查詢、減少不必要的 Agent 步數與工具呼叫、精簡 context 只放必要資訊、非即時任務用批次處理。我習慣先量測「錢花在哪一步」，再針對最貴的環節優化，而不是憑感覺。

Q19 你怎麼測試一個非確定性 (non-deterministic) 的系統？

傳統軟體測試是「輸入 A 必得輸出 B」，但 Agent 不一定。所以改用「評估而非斷言」：準備一組代表性測試案例與期望標準，跑多次看通過率與分數分布，設定可接受的門檻。像體育評分不是非黑即白，而是看「穩定落在幾分以上」。

F 落地與跨部門協作

Q20 公司是新導入 AI，你進來第一個專案會怎麼挑？

我會挑「高頻、痛、但風險低」的場景先做。高頻代表做出來價值大、容易被看見；風險低代表就算 Agent 出錯也不會釀大禍，適合邊做邊建信任。像新醫生先從常見小病累積口碑，而不是一進來就開大刀。第一個成功案例對「新導入」的長期推動至關重要。

Q21 業務或廠務同仁不信任 AI、覺得會出錯，你怎麼推動落地？

技術只是一半，信任是另一半。我會：先讓 Agent 當「副駕駛」而非「自動駕駛」——人保有最後決定權；公開展示它的依據（可追溯來源）；從一個願意嘗試的小團隊做出成果再擴散。讓人感覺「它在幫我、不是要取代我」，採用率才上得來。

Q22 製造業有哪些流程適合用 Agent？舉具體例子。

例如：用自然語言查詢「某客戶某料號的歷史品質異常」（跨系統撈資料的查詢 Agent）；自動彙整多廠的日報 / 品質報告成摘要；業務報價時自動比對歷史案例與料況輔助；供應鏈料況的問答助理。共同點是「資訊散落多系統、人工彙整很花時間」——這正是 Agent 的甜蜜點。

Q23 資料安全與機敏性怎麼考量？

製造業有客戶資料、製程 know-how 這些命根子。我會優先考慮地端或私有部署的開源模型（如 Ollama、Qwen、Phi 系列），敏感資料不外流到外部 API；做好權限控管（誰能問什麼）；以及對輸出做稽核紀錄。在 trade-off 上，內部敏感場景我寧可犧牲一點便利換安全。

G 行為面（對準 HR 的「完整度」訊號，請填入真實案例）

Q24 請舉一個你從頭到尾負責、且維護很久的專案。（幾乎必考）

講 DocAI：從架構選型（為什麼是 Milvus / FAISS 雙架構、切分策略怎麼定）、到多格式支援、跨機器知識共享的擴充，是一個持續迭代、長期維護的系統。重點要帶出「我做過哪些關鍵取捨、踩過哪些坑、後來怎麼優化」——因為這題本質是在驗證你拔長專案的能力，不是炫功能。

Q25 你提到 TeacherAssist.AI 經歷過完整重構，當初為什麼決定重構？學到什麼？

講從舊引擎遷移到新生成架構、前後端重新設計。重點放在「決策的延續性與架構彈性」：當初哪個架構決策後來變成包袱、所以這次怎麼設計才不會重蹈覆轍。能講出「重構的判斷依據與代價評估」，會讓人相信你懂長專案的真實重量。

Q26 你資歷 17 年，遠超過這個職缺要求，為什麼想加入一家製造業做內部 AI 落地？（化解 overqualified）

誠實但有方向地答：我這幾年專注 LLM 落地，最有成就感的不是做炫技 demo，而是把技術變成真正有人用、能持續運作的系統。製造業的場景具體、痛點明確、資料扎實，正好適合做「真正落地」的 AI。而且這是從 0 建起的機會，能定義藍圖、長期把它養大，這跟我重視踏實、長期投入的工作習慣很契合。把「太資深」重新定義成「我能扳拓荒 + 我懂長跑」，威脅就變成了賣點。

Q27 這領域變化很快，你怎麼保持技術更新？

我不是只看新聞，而是動手實作——平時會實際跑 LangChain、LangGraph、DSPy 這類框架，也會復現論文、玩開源模型。比喻是「不是看食譜就以為會做菜，要真的下廚才知道火

候」。對長期投入的職位，這種持續動手的習慣比一時的熱度重要。

Q28 如果主管交辦的方向你技術上不認同，你會怎麼處理？

我會先確認彼此在解決同一個問題，再用「數據與取捨」溝通而非堅持己見——例如做個小 PoC 用結果說話。如果評估後決策仍不同，在我充分表達後，我會 disagree and commit，把它做好。長專案最怕的是內耗，方向定了就全力執行，但前提是該講的我有講清楚。

Q29 你預期這個職位最大的挑戰是什麼？

坦白說有兩個：一是「新導入」意味沒有現成基礎與前例，要自己建標準、也要管理組織對 AI 的期待（避免被當萬靈丹）；二是 LLM 的不確定性跟製造業要求的穩定可靠之間的張力。但這兩個挑戰也正是我想來的原因——把不確定的技術做成企業敢用的系統，是我覺得最有價值的工程問題。

Q30 你有什麼問題想問我們？（務必準備）

反問展現你在想「落地」而非「面試」：例如——「公司目前內部最想優先 Agent 化的流程是哪個？」「AI 這塊目前是獨立團隊還是隸屬 IT？未來規劃多大？」「成功的標準會怎麼定義——是省下工時、還是先建能力？」這些問題會讓主管覺得你已經站在「進來要怎麼做事」的角度思考了。